

# CH4、Process

## Process 之管理

### 目錄：

Process 定義、與 Program 比較

PCB 內容

Process State Transition Diagram(STD)

Scheduler 種類(短、中、長期)

Context Switching

Dispatcher、Dispatch Latency

Process Control Operation(以 UNIX System Call 為主的程式追蹤)

評估 CPU Scheduling Performance 的 5 個 Criteria

各種 CPU 排班法則介紹(7+1 種)及相關名詞：Starvation、Aging、Convoy

Effect、Preempting and Non-Preempting

特定系統排班設計-Multi-Process System、Real-Time System(Hard 與 Soft)

### Process

(一)Def：A program in execution[忍]，也稱 job、task…等

Process 建立後，其主要組成有：

1. Process No. (ID)
2. Programming Counter (PC)：內放下條指令位址
3. Process State
4. Code Section
5. Data Section
6. Stack
7. CPU Register value

是 OS 分配資源之對象單位(Thread 是 OS 分配 CPU Time 的對象單位)

### Process in memory

```
int i;

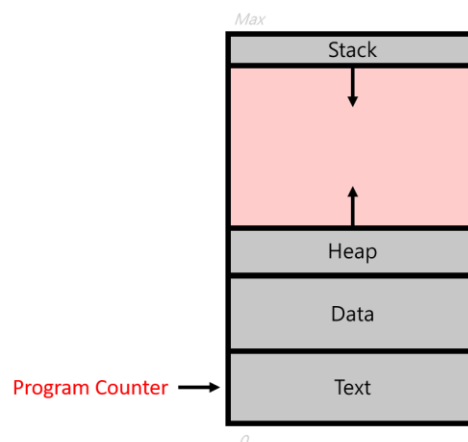
int foo(int x)
{
    static p;
    int *y;

    i=0;
    y=(int *)malloc(100);
}
```

*x、y 皆位於 stack*

*i、p 皆位於 Data*

*100 個記憶體位址位於 Heap*



## (二)與 Program(程式)比較

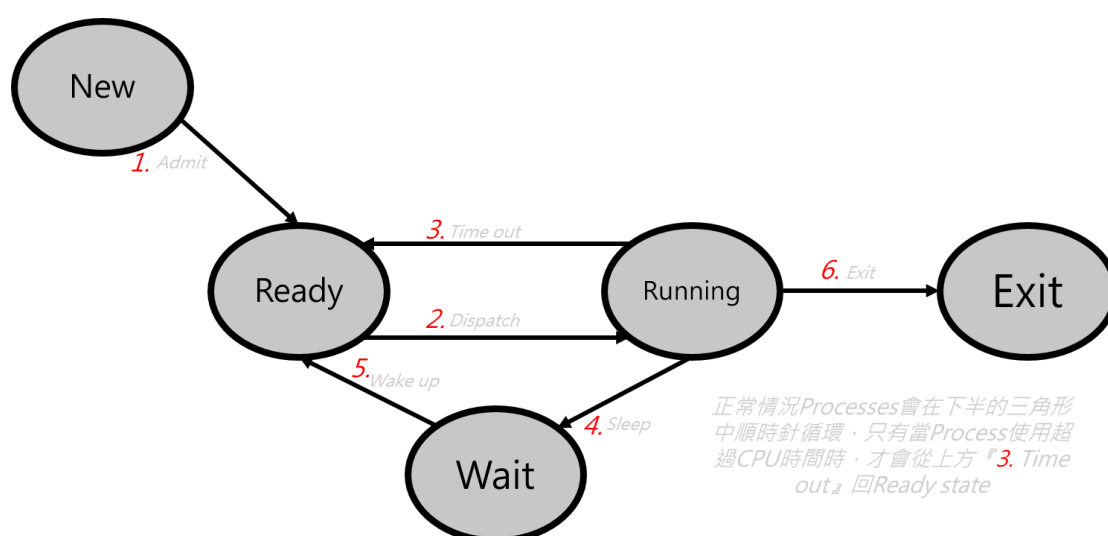
|                   |                                     |
|-------------------|-------------------------------------|
| Process           | Program                             |
| 執行中的程式            | Just a File store in storage device |
| “Activity” entity | “Passive” entity                    |

## Process 的 State Transition Diagram(STD)

(一)目的：描述 Process 之 life cycle

(二)版本： 1.[悉]5 個 State 之 STD  
2.7 個 State 之 STD[Stalling]  
3.UNIX STD[Stalling]

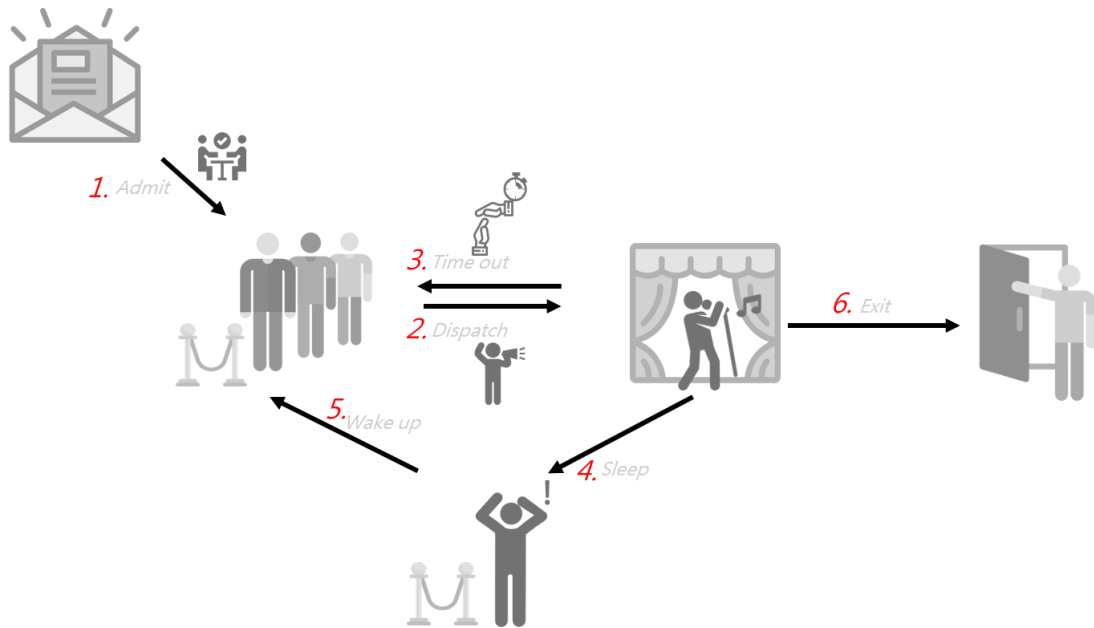
## 5 個 State 之 STD



| 狀態                             | 說明                                                                                                                        |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| New(Create)                    | Process 被建立，已分得 PCB 空間，尚未載入記憶體(or 尚未取得記憶體資源)                                                                              |
| Ready                          | Process 已在記憶體中，且在 Ready Queue 內，具有資格爭奪 CPU                                                                                |
| Running                        | Process 取得 CPU 執行中                                                                                                        |
| Wait(Block)                    | Process 在 Waiting Queue 中 wait for I/O completed or event occurs(不會與其他 Process 爭奪 CPU)                                    |
| Exit(Terminal、Zombie、Abort 少用) | Process 完成工作，正常結束或異常終止，此時可能其 PCB 尚未回收，因為要等其父親(Parent Process) collect 該子 Process 之成果後，才會回收 PCB space，其他資源(記憶體、CPU、I/O)已回收 |

| Transition | 說明                                                                          |
|------------|-----------------------------------------------------------------------------|
| Admit      | 當 Memory Space 足夠時，可由 Long-Term Scheduler (in Batch System)決定將此 Job 載入到記憶體中 |

|               |                                                                                                      |
|---------------|------------------------------------------------------------------------------------------------------|
| Dispatch      | 由 Short-Term Scheduler (CPU Scheduler)決定讓高優先權 Process 取得 CPU                                         |
| Time-out      | 也叫 Interrupt，執行中的 Process 會因某些事件發生，而被迫放棄 CPU，回去 Ready Queue。ex：Time-Out、Interrupt、高優先權 Process 到達、插隊 |
| Wait          | 叫 wait for I/O-Completed 或 event occurs                                                              |
| I/O-Completed | I/O-Completed 或 Event occurs                                                                         |
| Exit          | Process 完工或是異常終止                                                                                     |

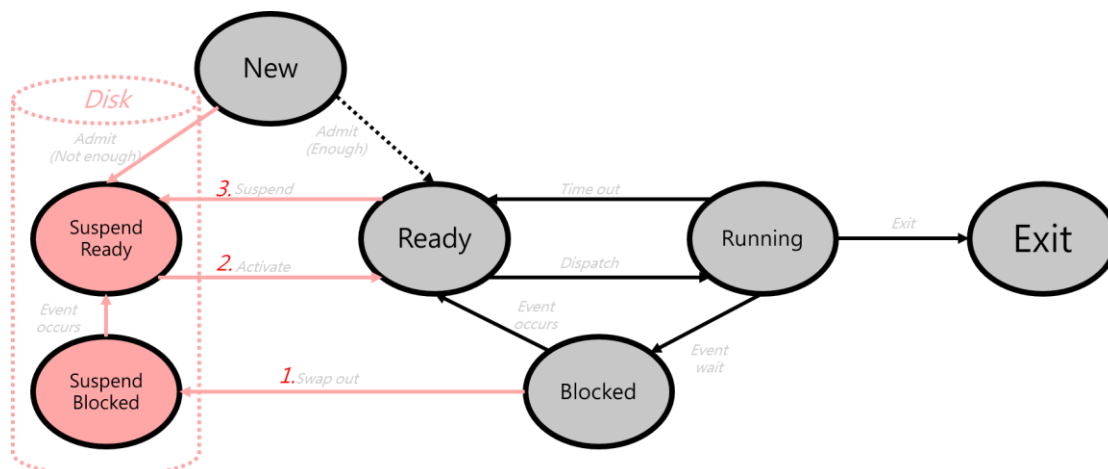


### 比喻：選秀節目

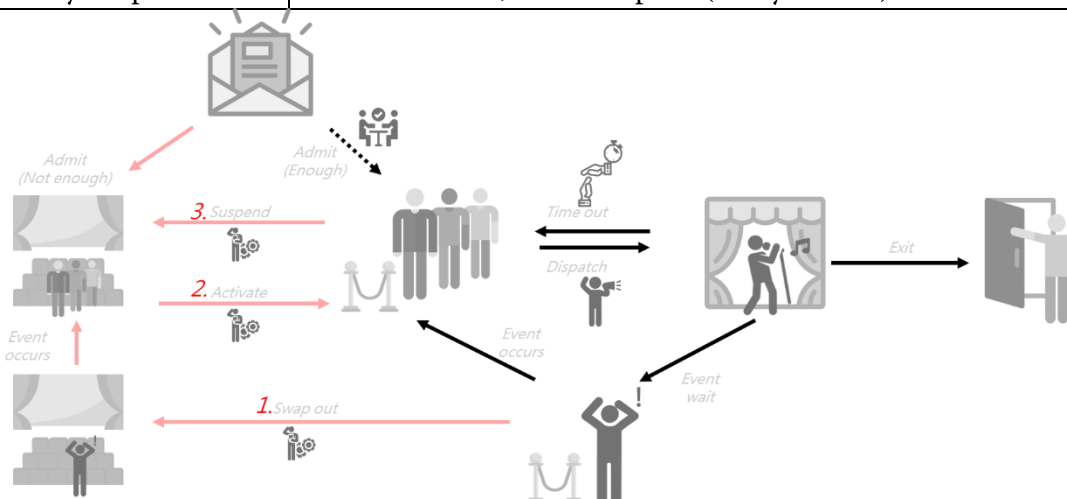
1. **New**：收到報名通知，被允許(**Admit**)進到表演舞台後場(**Ready**)
2. **Ready**：表演舞台後場有許多人正在等著上台表演(**Running**)，輪到的人，會依現場狀況，調整次序(例如可能不會讓同類型(ex：連續的 CPU Bound)表演連著上台、評審、觀眾會疲乏)上台表演(**Dispatch**)
3. **Running**：一次只能有一個人在台上表演，表演全部完成就離開(**Exit**)
4. **Wait**：若是表演過程中出現忘詞、走音...等出包狀況(**IO event**)，主辦單位非常好心，都允許回到後台準備好後(**IO Completed**)，再行等待下次上台機會(**Ready**)；而如果表演時間過長(**Time out**)，主辦單位也會請正在表演者先暫停(**Time out**)，一樣回到後台休息一下(**Ready**)、準備下一次的上台表演
5. **Exit**：離開後就結束本次選秀表演

## 7 個 State 之 STD

多增加了『當 Memory 不足時』的狀況



|               |                                                          |
|---------------|----------------------------------------------------------|
| Block Suspend | Process 被 swap out 到 Disk 中暫存，即 Blocked (asleep) in Disk |
| Ready Suspend | Event occurs 或 I/O-Complete (ready in disk)              |



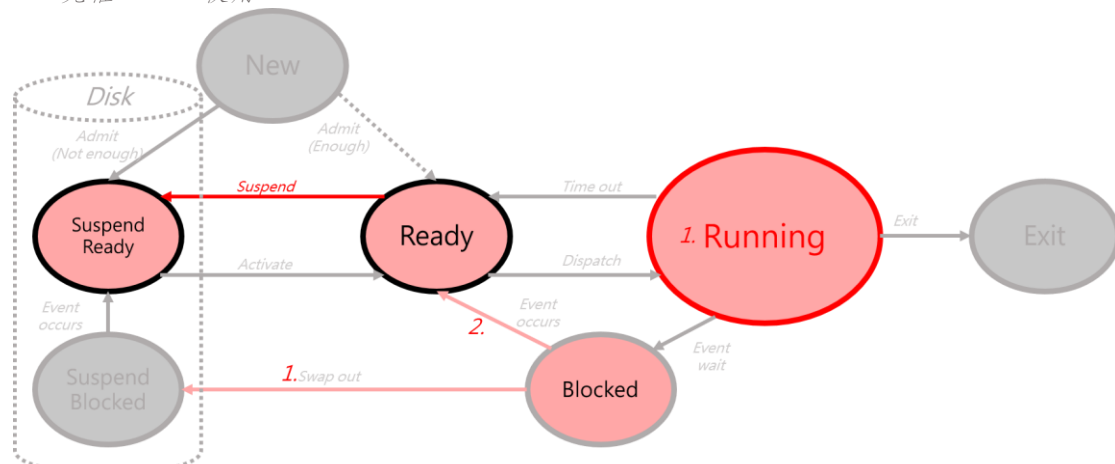
### 比喻：選秀節目

當表演舞台後場排隊等待上台的人太多的時候，便會請其他也要進到舞台後場等待上台，但較晚到達的表演者，先到台下休息，以避免後場人數過多的狀況發生

1. **Ready Suspend**：後場空間不足時，可以讓表演者休息等後的台下空間
2. **Block Suspend**：因為後場空間不足，讓需要解決突發狀況的表演者，一樣到台下休息並解決狀況，

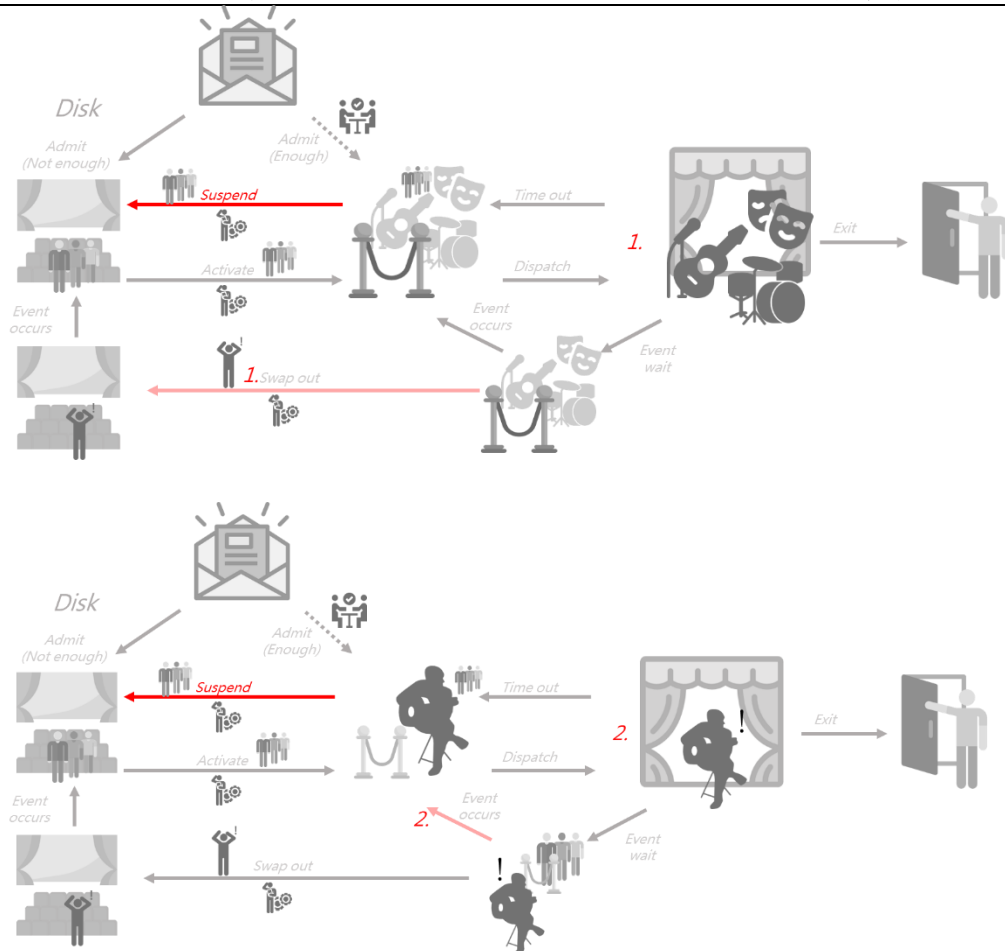
|                    |                                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Suspend (swap out) | 當記憶體空間不足，又有其他高優先權 Process 需更多記憶體空間，會由 Medium-Term Scheduler 決定將 Blocked Process swap out 至 Disk，以空出記憶體空間                                |
| Activate (swap in) | 當記憶體空間有空位，Medium-Term Scheduler 可將它們 swap in 回記憶體中 ready for execution                                                                  |
| Suspend (swap out) | 支持此 transition 之理由有二(見下方註解)：<br>1. 所有 Blocked process 皆 swap out 後，記憶體空間仍不足時<br>2. 所有 Blocked state process 之優先權皆高於 ready state process |

1. This is a poor design, 但仍可支持, 理由如下: 若所有 Block Suspend State 之 Process 優先權皆高於 Ready Suspend Process, 且 OS believe them will become ready soon.
2. It's also a poor design, 但可支持之理由如下: 若有一個高優先權 process 從 Block/Suspend 變成 Ready/Suspend 時, 則 OS 可以強迫低優先權之執行中 Process, 放掉 CPU、記憶體, 供高優先權 Process 使用



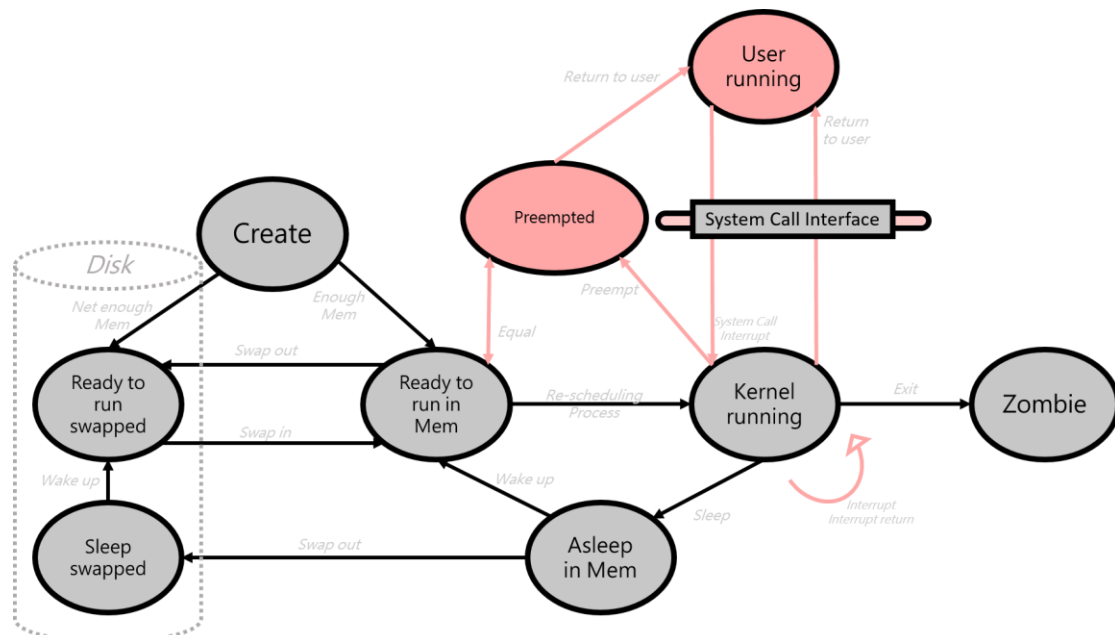
### 比喻：選秀節目

1. 若今天有一正在表演者的道具過多，舞台上都已經無法容納，則會將其道具往後台移動，又後台空間有限，故請一些正在後台等待上台的表演者先至台下休息等待
2. 本來在台上表演的表演者突然出狀況，但因為他是超級大咖，所以主辦單位會優先讓他到後場準備，哪怕是他還沒解決突發問題，依然相信他隨時都能解決問題、準備上台；但又因後台空間有限，故請一些正在後台等待上台的表演者先至台下休息等待



## UNIX STD[Stalling]

多增加了『Running 細分為 kernel mode 與 user mode』的狀況



Scheduler 種類：

### 1. Long-Term Scheduler：

(一)Def：又叫 Job Scheduler，目的是從 Job Queue 中挑選一些 Jobs 載入到記憶體中

(二)特點：

1. 執行頻率最低(目前的個人電腦中幾乎沒有，通常是 Server 才有)
2. 可以調整 Multiprogramming Degree(在 Memory 中等待執行的 Process 數量)
3. 可以調控 I/O Bound Job 與 CPU Bound 之混合比例
4. Batch System 採用，但是 Real-Time 及 Time-Sharing System 不採用

#### **I/O、CPU Bound(又稱 IO、CPU Intensive)**

##### 1. I/O Bound

Def：此類之工作大都是需要大量的 I/O operation (resource)，但對於 CPU Time (computation) 需求很少，所以其工作效能受限於 I/O Device 速度。

ex：DBMS 財報列印

##### 2. CPU Bound

Def：與 I/O Bound 相反

ex：氣象預估，科學模擬

### 2. Short-Term Scheduler

(一)Def：又叫 CPU Scheduler 或 Process Scheduler，目的是從 Ready Queue 中，挑出一個高優先權 Process，分派 CPU 給它執行

(二)特點：

1. 執行頻率最高
2. 無法調整 Multiprogramming Degree
3. 無法調控 I/O Bound Job 與 CPU Bound 之混合比例
4. 所有 System 皆採用

### 3. Medium-Term Scheduler

(一)Def：Time-Sharing System 採用目的：當記憶體空間不足且又有其他高優先權 Process 需要記憶體空間時，此 Scheduler 會啟動，它會挑選一些 Process (ex：blocked process、低優先權 Process) 將其 swap out 到 Disk 中保存，以空出記憶體空間，供其他 Process 使用。將來等到有足夠的記憶體空間 released 後，此 Scheduler 可再將它們 swap in 回記憶體，ready for execution

(二)特點：

1. 執行頻率三者居中
2. 可以調整 Multiprogramming Degree
3. 可以調控 I/O Bound Job 與 CPU Bound 之混合比例
4. Time-Sharing System 採用

#### 比喻：選秀節目

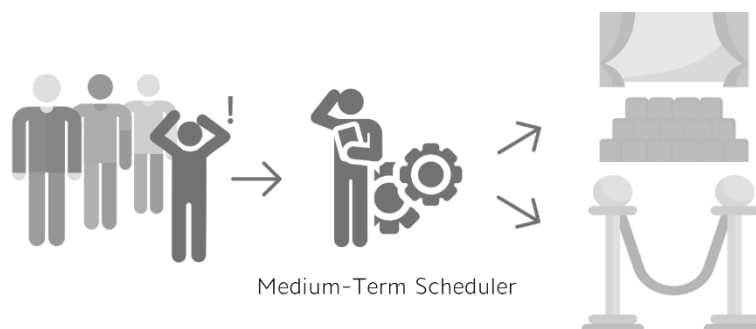
1. Long-Term Scheduler：報名單位，允許有表演的機會，可略，路過的人不用報名、直接走進去等待表演也可以
2. Short-Term Scheduler：舞台助理，最重要，每個舞台都一定要有舞台助理
3. Medium-Term Scheduler：台下引導人員，安排台下坐位與後台之間的調度  
後台大小是有限的，所以進去等候的表演者表演類型固定而無法輕易改變，舞台助理(Short-Term)無法調度(CPU Bound 與 IO Bound)，但報名單位(Long-Term)與台下引導人員(Medium-Term)，可以決定喜劇與悲劇的平衡數量



Long-Term Scheduler



Short-Term Scheduler



Medium-Term Scheduler

## Process Control Block(PCB) 內容

(一)Def：OS 為了管理所有 Process，會在 kernel memory 中，替每一個 Process 各自準備一個 Block(表格)，記錄 Process 之所有相關資訊。PCB 主要內容有：

1. Process No. (ID)：是 unique 的(唯一的)
2. Process State：ex：Ready、Running、Waiting
3. Programming Counter：內放下一個要執行的 instruction's address
4. CPU Register：ex：記錄使用到暫存器的值。ex：Accumulator、Password、Stack、Top
5. CPU 排班 information：ex：Process priority、Arrived time、CPU time quantum...
6. Memory management information：隨 OS 記憶體管理方法不同，記錄不同資訊。ex：Base/Limit Register 或 Page Table 或 Segment Table
7. Accounting information：ex：Process 已使用多少 CPU time。哪些資源、還剩多少 CPU Time 可用

Note：目的： 1.記帳收錢

2.Administrator 調整效能依據

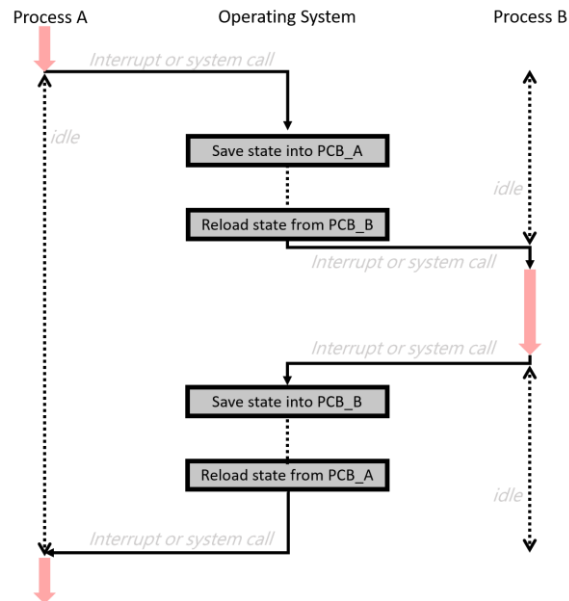
8. I/O status information：ex：Process 已發出多少 I/O-request，完成狀況如何、佔用哪些 I/O 資源(目前)

| Process ID          | Process State |
|---------------------|---------------|
| Program Counter     |               |
| CPU Register        |               |
| CPU Scheduling info |               |
| Memory info         |               |
| Accounting info     |               |
| IO info             |               |

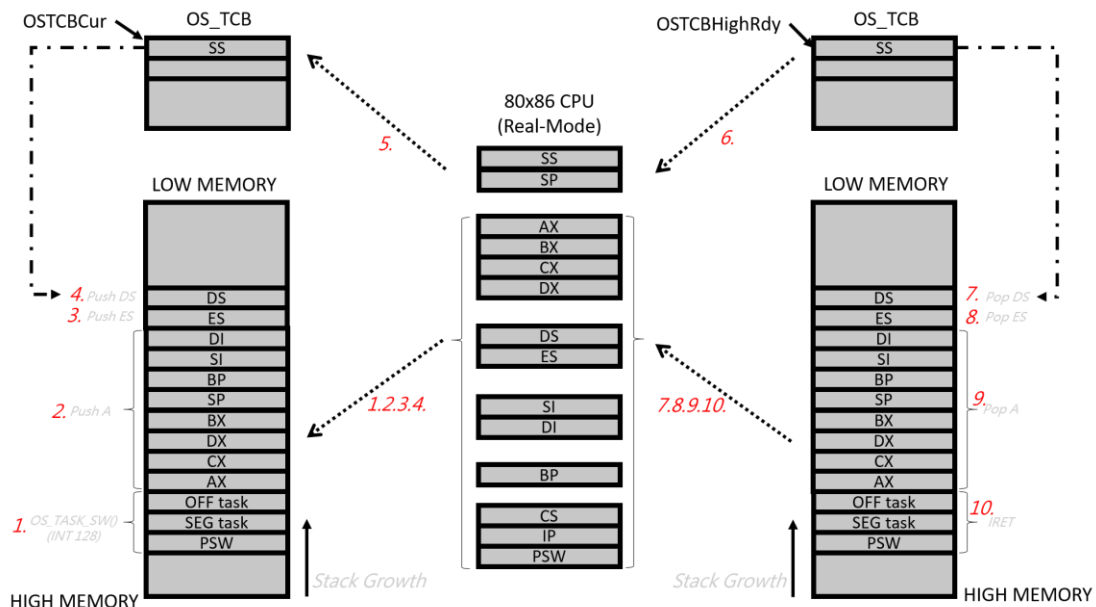
## Context Switching

(一)Def：當 CPU 要從 Running Process 切給另一個 Process 使用之前，kernel 必須“保存(store)”Running Process 之前狀態 (ex：PC 之值、Stack、CPU Register，即存回此 Process 之 PCB)，且要載入(restore)另一個 Process 的狀態資訊 from Process PCB。此一工作稱為 Context Switching.

Context Switching 是一額外負擔，其時間長短大都取決於 HW 因素。ex：Register 數目、記憶體空間存取指令速度...



Process 切換時序



Context Switching 時，PCB 記憶體轉移示意圖

## (二) 如何降低

- [法一]：如果 Register 數目很多，則可讓每個 Process 皆有自己的(private) "Register Set"，則 OS 只要切換一個指標指向另一個 Process 的 Register Set 即可完成 Context Switching without memory store/restore。因此 Speed fast。
- [法二]：(法一的改良型，比較針對 System Call 與 Interrupt)讓 System Process 及 User Process 各自擁有自己的 Register Set，如此一來，兩者之切換只要改變指向 Register Set 的 pointer 即可
- [法三]：使用 Multi-Threading 機制，Thread 共用資源(Memory)
- [法四]：提供更快速的特殊指令

### 比喻：選秀節目

不同表演者切換時，需有時間調整舞台，其節省時間方式如下：

1. 旋轉舞台：多個舞台讓表演者準備，要表演時只需要**旋轉至該舞台**即可直接開始
2. 針對特定表演，例如歌劇的第一幕、第二幕，不同場景切換時，**保留布景**，只切換舞台
3. 共用道具：讓同類型的表演者**共用道具**，就不用搬上搬下(ex：打擊樂器)
4. 請專業**快速的搬運工**

### Dispatcher(分派器)及 Dispatch Latency(分派延遲)

(一)Def：Dispatcher，此一模組的目的是要將 CPU 控制權授予經由 CPU Scheduler 所選出之 Process

- 主要工作有三：
- 1.Context Switching
  - 2.Charge mode from kernel mode to user mode
  - 3.Jump to the execution entry of the selected process

上述這些工作的時間總和稱為 Dispatch Latency

(二)希望 Dispatch Latency 愈短愈好，這些 process 可以盡早開工

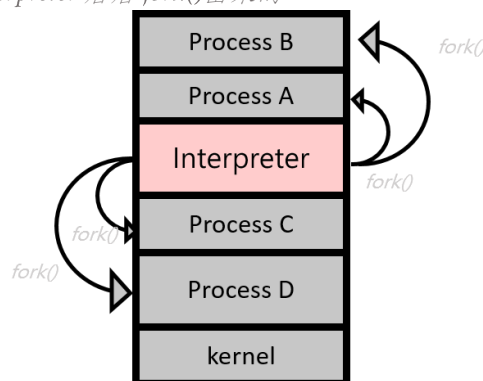
### 比較：

1. *Interrupt Latency*：包括在呼叫 Interrupt Service Handler 之前、儲存目前 Process 狀態的時間，包含下列工作：決定 Interrupt 時間、執行 Context Switch、跳到 ISR 執行
2. *Dispatch Latency*：停止一個 Process 並開始執行另一個 Process 所需要的時間，包含：搶先目前在 Kernel 執行的 Process、釋放出即將執行的 Process 所需的資源、Context Switch 給新的 Process

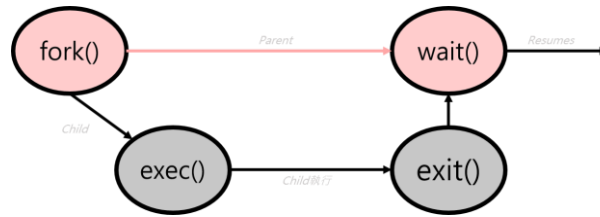
### Process Control Operations

#### Part 1：理論部分

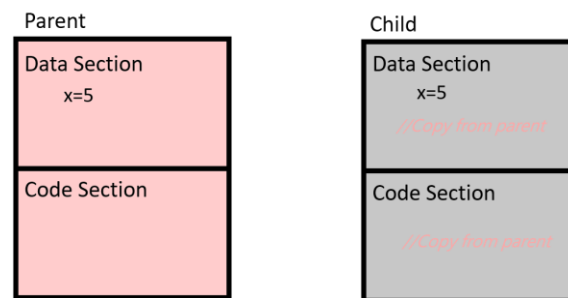
1. 意指：Process 建立、終止、暫停、恢復執行，設定/修改/讀取 Process attributes 值…等。上述這些皆是 OS 應該提供的服務
2. Process 是可以建立自己的 Child Process，目的是要 Child Process 作工作  
*//所有的 Process 都是由 Interpreter 層層 fork() 出來的*



3. Child Process 所作的工作可分為 2 類：
  - (1) 與 Parent 相同的工作(純 fork() 出來的 Child)
  - (2) 特定的工作(與 Parent 不同，有經過 execvp())
4. Parent 與 Child 之互動關係
  - (1) Concurrent execution(通常工作相同)
  - (2) 父等子 until child terminated(有使用 wait())



5. Child Process 所需之資源由何提供
  - (1) OS 提供(可能會限制 child 數)
  - (2) Parent 提供
6. Parent 若終止，則 Child Process 會如何處理？
  - (1) 一併終止，稱 cascading termination
  - (2) child process 存活：
    - 1.向 OS 取得資源
    - 2.向祖先取得資源(通常會被 inetd，此一 process 收養)



## Part 2：以 UNIX System Call 為例

1. fork()：此 System Call 建立 child process，而 fork()之傳回結果，對象如下：
  - (1) 失敗：因為資源不足、無法建立，會傳回負值(-1)給 OS，then pass to parent process
  - (2) 成功：會回傳：
    - 1.零值：給 child process
    - 2.大於零值：給 parent process，且此值為 child process ID
2. vfork()：與 fork()功能類似，差別在於子與父共用 stack，即子直接延伸使用父之 stack，故資料可能會同時修改

### 求結果為何？

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int x=0;
int main()
{
    pid_t pid;
    pid= vfork();          /* fork another process */

    if (pid< 0)            /* error occurred */
    {
        fprintf(stderr, "Fork Failed");
    }
}

```

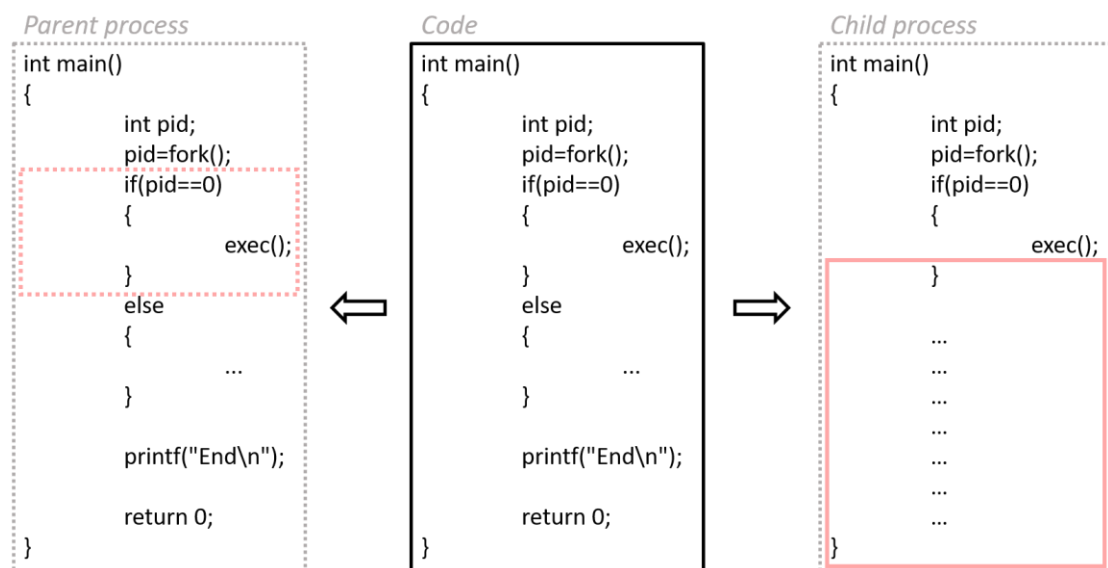
```

        exit(-1);
    }
    else if (pid== 0) /* child process */
    {
        x++;
        exit(0);
    }
    else /* parent process */
    {
        wait (NULL); /* parent will wait for the child to complete */
        printf("%d",x);
        exit(0);
    }
}

```

*x=1 : 因為父、子行程共用 Stack*

3. `getpid()` : 此 System Call 用以取得 Process ID
4. `execvp()` : 或 `exec()`、`execve()`。此 System Call 用以載入特定的 binary code(可執行檔)執行，並覆寫掉 `exec` 之後子行程之程式碼(ex : `execvp("目錄名稱", "檔名", "參數")`)。實際流程如下： 程式在 `fork` 之後，子行程會擁有和父行程一模一樣的 data section 與 code section，當 "`if(pid==0) exce...`" (或任何 `exce`)之程式碼出現，則會載入一段新功能的 code。此時子行程就會覆寫掉所原本 copy 來自父行程、`exec` 之後的程式碼。如下圖：



中間為原始程式碼。父行程會跳過子行程的部分(圖左粉紅虛線區域)直接執行後半；子行程則因 `exec`，會洗掉後面所有程式碼(圖右粉紅實線區域)

5. `wait()` : 此 System Call 用以暫停 process execution 直到某個事件發生。ex : 父等子 until 子終止
6. `exit()` : 此 System Call 用以終止 process 的執行、回收資源，通常 `exit(0)`表正常終止；`exit(-1)`表異常終止(通常可以不寫，因為右括號"}"常常是已經內建了)

Note :

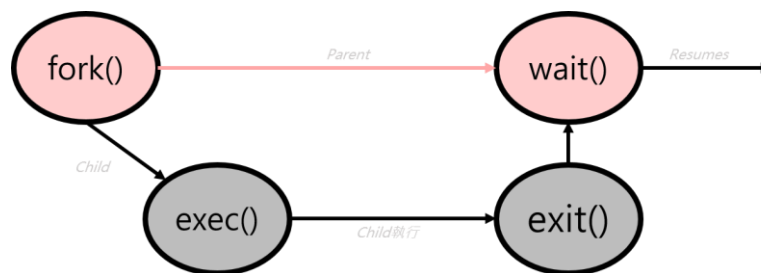
1. 若子所作之工作與父相同，則直接 `fork()` 完，就已經達成目的
2. 若子要作特定工作(與父不同)，則子必須執行 `execlp()` 此 System Call，載入特定工作執行檔

OS 會配置 `child process` 記憶體空間，與 `parent` 是佔用不同的記憶體空間，且子的 `Data Section` 及 `Code Section` 內容均來自父之 `copy/initially`

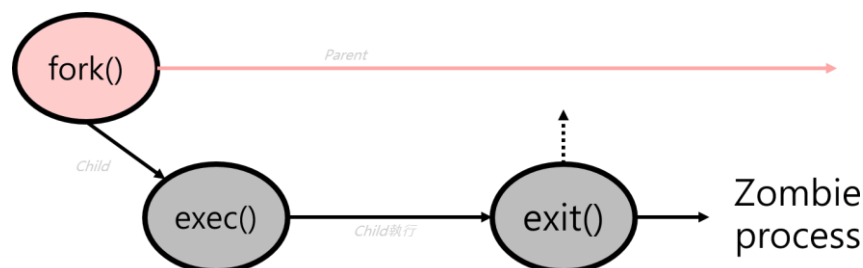
## Process Terminal

綜合上述 4、5、6，若父、子行程完成任務的時間有時間差，則會有以下三種情況出現

1. **Normal**：一般來說，父行程會等待子行程完成任務，當子行程任務結束前，會呼叫 `exit` 來通知父行程自己已經結束。呼叫後，`memory`、`CPU` 資源都會釋放，但 `PCB` 還不會釋放(此時的狀態稱做 **Zombie process**)，會需要等到父行程呼叫的“`wait()`”，等到子行程結束，才會將子行程的 `PCB` 釋放



2. **Zombie**：若父行程一直不呼叫(或本來就沒有要呼叫)`wait()`，子行程便成為只剩 `PCB` 的 **Zombie process**。遇到此種狀況，Linux 系統會將 `root process` 設定為子行程的『新·父行程』，而 `root process` 會週期性的呼叫 `wait()`，以回收子行程之 `PCB`。



### A Zombie Child Process

```
#include <stdio.h>
#include <sys/types.h>
```

```
int main()
{
```

```

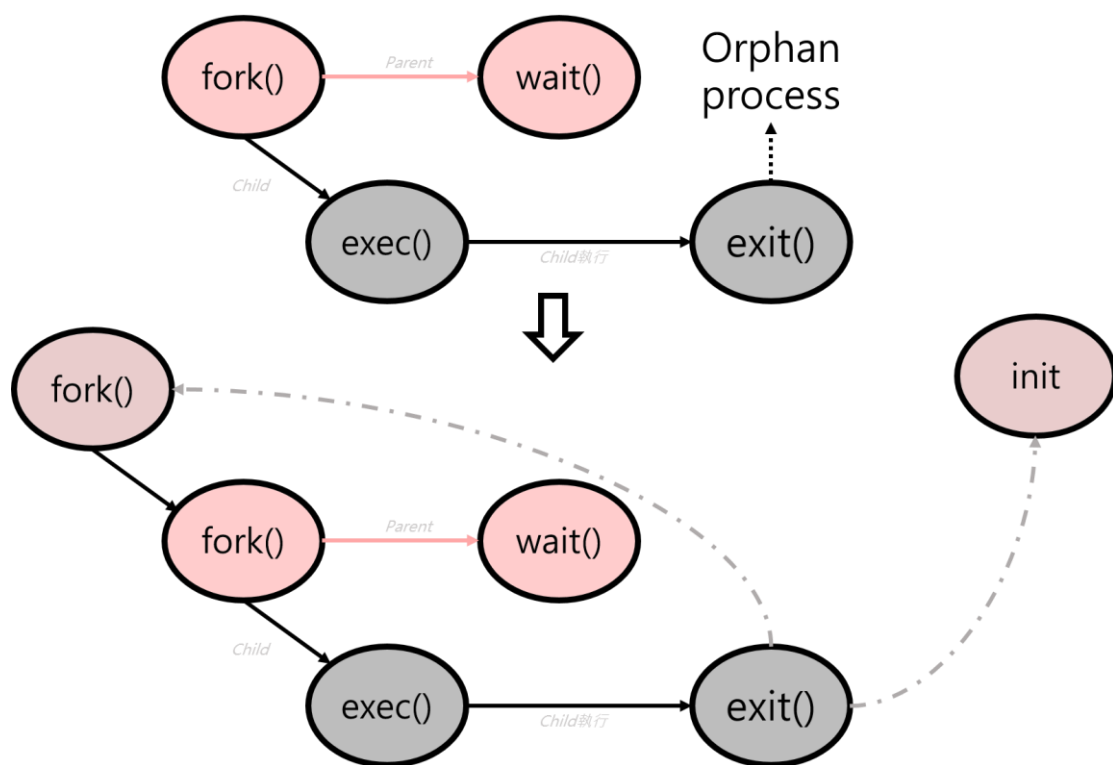
if (pid == 0) /* child process */
{
    printf("child process = %d\n", getpid());
    exit(0);
}

else /* parent process */
{
    sleep(20); // let the child print the message
    printf("parent pid=%d\n", getpid());
    exit(0);
}
}

```

*After the child terminates, it becomes a zombie until being adopted and read by init.*

3. Orphan：以上兩種情況都基本上是子行程後完成，但若是父行程已然完成並結束，子行程尚未結束；等到子行程結束時，已經沒有父行程可以接收其 PCB，則稱為 Orphan process。遇到此種狀況，子行程可以接到祖父行程(grand-parent process)、或者”init” process (process 0)上。



### Part 3：程式追蹤

例 1：建立 Child Process 執行 ls 命令檔，且父等子完成後才列印“Child Completed！”

```
int main()
{
    pid_t pid;
    pid=fork();    //Child Process 從此行誕生
    if(pid<0)      //當錯誤發生
    {
        printf(stderr,"Fork Failed\n");
        exit(-1); //表示異常終止
    }
    else if(pid==0)//Child Process
    {
        execlp("/bin/ls","ls",NULL); //execlp() 後程式碼都將被洗掉，故 Child 不執行
    }
    else //Parent Process
    {
        wait(NULL);
        printf("Child Completed\n");
        exit(0);    //表正常終止
    }
    //Parent ㄓ Child 都要執行之內容;
}
```

例 2：

假設 Parent、Child 的 ID 值分別為 2600 與 2603，求下列程式結果

```
int main()
{
    pid_t pid, pid_1;
    pid=fork();    //Child Process 從此行誕生
    if(pid==0)     //Child Process
    {
        pid1=getpid();    //取得自己的 Process ID
        printf(pid);      //A
        printf(pid_1);    //B
    }
    else //Parent Process
    {
        pid1=getpid();
        printf(pid);      //C
        printf(pid_1);    //D
    }
}
```

結果為：A=0、B=2603、C=2603、D=2600

例 3：

**求 A 之值？**

```
int value=5;

int main()
{
    pid_t pid;
    pid=fork();    //Child Process 從此行誕生
    if(pid==0)    //Child Process
    {
        value+=5;
        return 0;
    }
    else //Parent Process
    {
        wait(NULL);
        printf(value); //A
        return 0;
    }
}
```

結果為：A=5

因為 Line\_09 有被加到 5 的，只是 Child Process 裡的 value，Parent 並沒有被加到

例 4-1：

**求結果為何？**

```
int main()
{
    pid_t pid;
    pid=fork();    //Child Process 從此行誕生
    if(pid==0)    //Child Process
    {
        printf("A");
    }
    else //Parent Process
    {
        printf("B");
    }
}
```

結果：AB 或 BA

例 4-2 :

**求結果為何？**

```
int main()
{
    pid_t pid;
    pid=fork();    //Child Process 從此行誕生
    if(pid==0)    //Child Process
    {
        printf("A");
    }
    else //Parent Process
    {
        printf("B");
        wait();
    }
    printf("C");
}
```

結果：ABCC、BACC、ACBC

例 5-1 :

**假設 count 是共用變數 by Parent 與 Child，初值=5**

```
int main()
{
    pid_t pid;
    pid=fork();    //Child Process 從此行誕生
    if(pid==0)    //Child Process
    {
        count++;
        printf(count);
    }
    else //Parent Process
    {
        wait();
        count--;
        printf(count);
    }
}
```

結果：65

例 5-2 :

假設 count 是共用變數 by Parent 與 Child，初值=5

```
int main()
{
    pid_t pid;
    pid=fork();    //Child Process 從此行誕生
    if(pid==0)    //Child Process
    {
        count++;
    }
    else //Parent Process
    {
        count--;
    }
    printf(count);
}
```

結果：45、65、55、66、44、46、64

前三結果好理解：

45=子執、子印、父執、父印

65=父執、父印、子執、子印

55=父執、子執(前兩可交換)、父印、子印(後兩也可交換)，共四種排序可能

66=子先於父一些，但近乎同時執行，子執行完時父還在執行，子印出前父執行完、卻復寫掉子原本的數值，故父、子印出時，皆為父執行完之結果

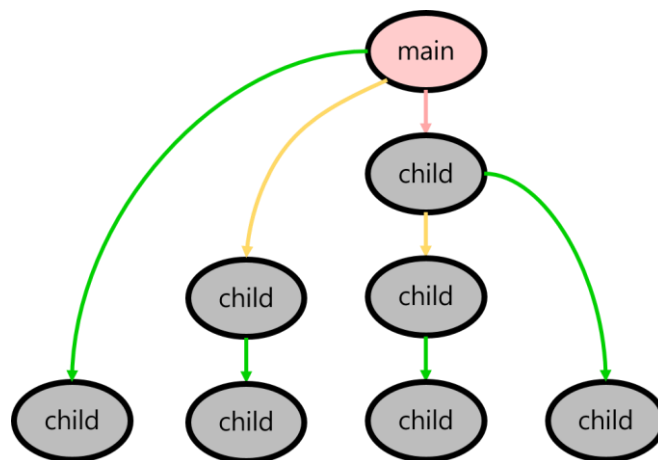
44=同理如下、只是父子交換而已

46=子先於父一些，但近乎同時執行，子執行完後馬上印出，但父還在執行並儲存，故子印出後父又復寫掉其值，而後印出的則為父之結果

64=同理如下、只是父子交換而已

56 例 6-1 :

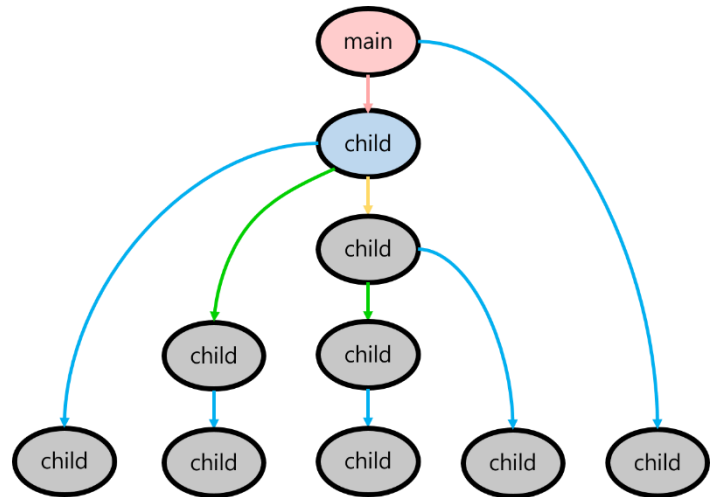
```
int main()
{
    fork();
    fork();
    fork();
}
8 個
```



例 6-2：

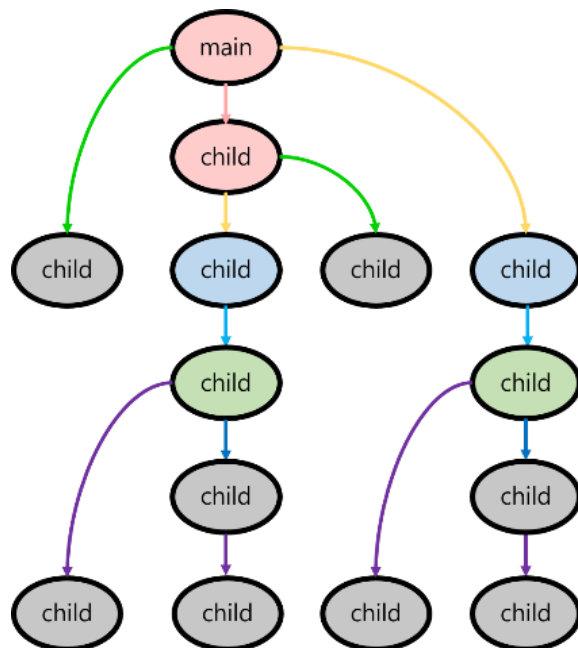
```
int main()
{
    if(fork()==0)
    {
        fork();
        fork();
    }
    fork();
}
```

10 個



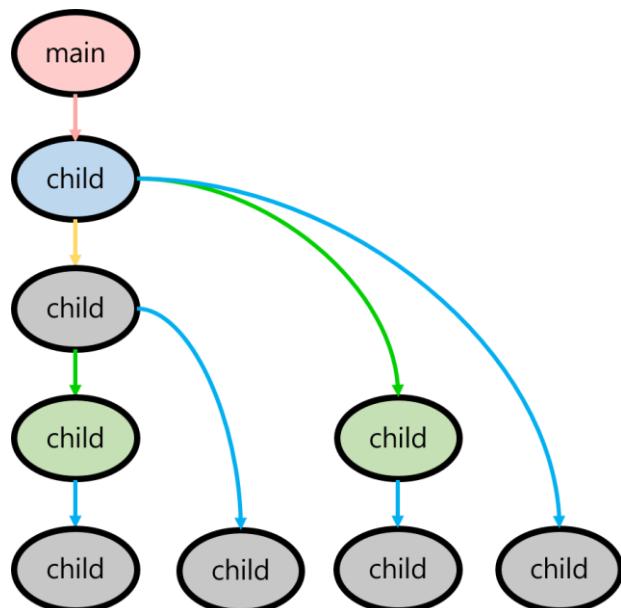
例 6-3：

```
int main()
{
    fork();
    if(fork()>0)
    {
        fork();
    }
    else if(fork()==0)
    {
        fork();
        fork();
    }
}
```



例 6-4：

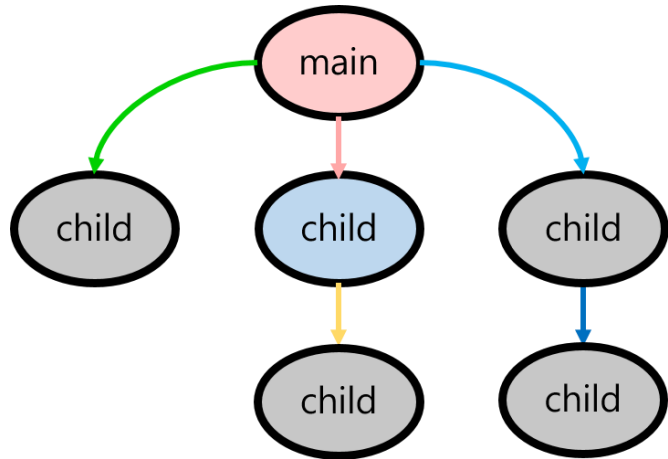
```
int main()
{
    for(i=0;i<3;i++)
    {
        if(fork()==0)
        {
            fork();
            fork();
            fork();
        }
    }
}
```



例 6-5 :

```
int main()
{
    for(i=0;i<3;i++)
    {
        if(fork()==0)
        {
            fork();
        }
        else if(fork(>0))
        {
            if(fork()==0)
            {
                fork();
            }
        }
    }
}
```

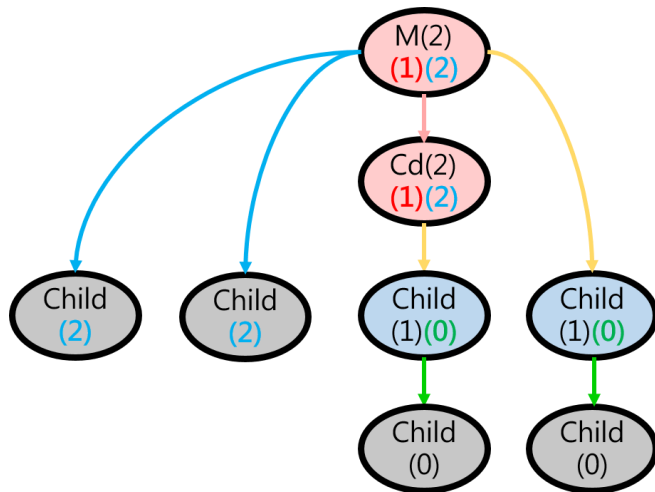
6<sup>3</sup>=216 個



例 6-6 :

```
int main()
{
    int a=2;
    fork();
    a--;
    printf(a);
    if(fork()==0)
    {
        a--;
        printf(a);
        fork();
    }
    else
    {
        a++;
        fork();
        printf(a);
    }
}
```

8 個 printf : 2 個 0 、 2 個 1 、 4 個 2



## 評估 CPU Scheduling 效能之 5 個 Criteria

1. CPU Utilization(CPU 利用度/率) =  $\frac{\text{CPU 花在 Process Time}}{\text{CPU total Time}}$   
 CPU total Time (=process execution + context switching + idle time +...)  
 例：process 平均花費 5ms 在執行、1ms 在 context switching  
 ⇒ CPU Utilization =  $5/(5+1) = 5/6$
2. Throughput 產能：單位時間內完成的 Jobs 數目
3. Waiting Time 等待時間：Process 花在 Ready Queue 中等待獲得 CPU 之等待時間加總  
 例：

| 動作 | Process 到達 | 取得 CPU 並執行 | Time out | 再度取得 CPU 並執行 | 完工 |
|----|------------|------------|----------|--------------|----|
| 時間 | 2          | 8          | 15       | 19           | 26 |

 ⇒ Waiting time =  $(8-2) + (15-8) = 6+4 = 10$
4. Turnaround Time 完成時間：從 Process 進入(到達)Ready Queue 到它工作完成的這段時間差值  
 承上例：  
 ⇒ Turnaround Time =  $26-2 = 24$
5. Response Time 回應時間：user (user program) input 命令/Data 給系統到 System 產生第 1 個回應時間差  
 \*Time-Sharing System、user-interactive app 特別重視  
 承上例：  
 ⇒ Response Time =  $8-2 = 6$

由上述可知，排程目標是讓 1、2 提高、3、4、5 降低；各項之間亦可能會衝突：Throughput vs response

## 7+1 種 CPU 排班法則介紹：

- |         |                       |             |
|---------|-----------------------|-------------|
| 1. FCFS | 4. Priority           | 7. MFQs     |
| 2. SJF  | 5. Round-Robin        | 8. HRRN(補充) |
| 3. SRTF | 6. Multi-level Queues |             |

### 1. FCFS

(一)Def：到達時間最小的 Process，優先取得 CPU

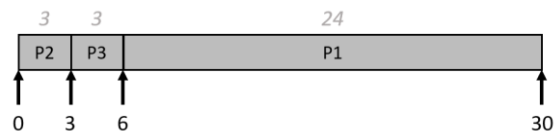
(二)例：

| Process | CPU (burst) Time |
|---------|------------------|
| P1      | 24               |
| P2      | 3                |
| P3      | 3                |

到達時間為 0、到達順序為：P1、P2、P3(依序近乎同時)

1. 畫出 Gantt Chart(甘特圖)
2. average waiting time ?
3. average turnaround time ?

1. 圖



2. Avg waiting time:  $[(0-0)+(24-0)+(27-0)]/3 = (0+24+27)/3 = 17$

3. Avg turnaround time:  $[(24-0)+(27-0)+(30-0)]/3 = 27$

(三)分析：

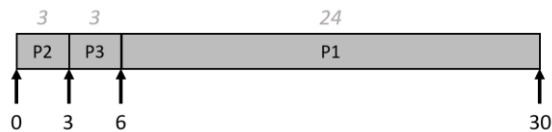
1. 排班效能最差，即 average waiting time、average turnaround time 最長，但簡單易製作(使用 Ready Queue -> FIFO)
2. 可能會有"Convoy Effect"護衛效應
3. 公平
4. No Starvation，沒有飢餓現象
5. Non-Preemptive 法則(不可插隊、不可搶先)

[名詞] Convoy Effect

(一)Def：許多 Process 均在等待一個需很長 CPU Time 之 Process 完成工作，才能取得 CPU 造成 average waiting time 很長之不良效應

例如上例，若到達時間改成 P2、P3、P1，則效率會大大提高：

1. 圖



2. Avg waiting time:  $[(0-0)+(3-0)+(6-0)]/3 = (0+3+6)/3 = 3$

3. Avg turnaround time:  $[(3-0)+(6-0)+(30-0)]/3 = 13$

[名詞]Starvation

(二)Def：Process 因為長期無法取得完工所需之各式資源，導致它遲遲無法完工，形成"Indefinite Blocking"現象(不確定何時結束、與 deadlock 的確定無法結束不同)。容易發生在不公平對待之環境中，若再加上"Preemptive"機制，則更容易發生

Note：類比於"中樂透"

(三)解決方案：Aging(老化)技術

Def：隨著 Process 待在 System 內的時間逐漸增加，我們也『逐步調高此 Process 的優先權』，經過一段時有限的時間，此 Process 會有最高優先權，故可取得 needed resources 完工，所以不會 Starvation

Note：Soft-Real-Time 不採用"Aging"技術(CH1)：Highest priority job 需要一直維持最高優先權

[名詞] Non-Preemptive 與 Preemptive

[版本一](白話文)

### 1. Non-Preemptive 法則

(1) Def: 除非執行中的 Process 自願放掉 CPU, 其他 Process 才有機會取得 CPU, 否則就只能 wait, 不可逕自搶奪 CPU

ex: 完成工作、wait for I/O-Completed after issue I/O request

(2) 優點: 1.Context Switching 次數較少

2.Process 完工的時間點, 比較可以預期(predictable)

3.比較不會有 Race Condition Problem (CH6)

(3) 缺點: 1.排班效能較差(因為可能有 Convoy Effect)

2.不適合用在 Time-Sharing System 及 Real-Time System

### 2. Preemptive 法則

(1) Def: 執行中的 Process 有可能『被迫』放棄 CPU 回到 Ready Queue, 將 CPU 切給別人使用。ex: Time-Out、Interrupt

(2) 優點: 1.排班效能較佳、平均 waiting、turnaround time 較小

2.適用於 Real-Time System 及 Time-Sharing System

(3) 缺點: 1.完工時間比較不可預期

2.Context Switching 次數較多

3.需注意 Race Condition 之發生(CH6)

註: Preemptive 未必不公平  $\Rightarrow$  RR: 不公平也未必 preemptive  $\Rightarrow$  SJF

[版本二]: 從 CPU 排班決策(啟動)之時機點作區分

1. Running -> Wait: wait for I/O Completed[自願]

2. Running -> Ready: Time-Out[被迫]

3. Wait -> Ready: I/O Completed[自願]

4. Running -> Exit: 完成工作[自願]

[註]高優先權 process become ready, OS 啟動 Scheduling 使之獲得 CPU; 低優先權 process 被迫放棄 CPU。所以若排班決策啟動點, 如果只包含 1、4, 未包含 2、3, 則稱此 CPU 是不搶先 Non-Preemptive 的, 或者稱之為合作 Cooperative 的, 否則為 Preemptive。

### Cooperative scheduling 合作式排程

優點: 易實作、且不需要額外硬體(ex: timer, 屬 preemptive 的 RR 才需要)

行程可能會自願放棄 CPU(呼叫 sleep(0)或 yield())

缺點: 回應差(poor responsiveness)

只需要一個有問題的行程, 就能讓整個系統崩潰

系統範例: Windows 3.1、Old versions of Mac OS、Sensor-node OS(ex: TinyOS)

Note:

凡是 xx -> Ready, 皆列入 Preemptive 元素(選項), 所以 Ready/Suspend -> Ready 與 New -> Ready 亦是

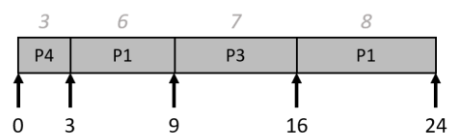
## 2. SJF

(一)Def：具有最小的 CPU Time 之 Process 優先取得 CPU

(二)例 1：

| Process | CPU (burst) Time |
|---------|------------------|
| P1      | 6                |
| P2      | 8                |
| P3      | 7                |
| P4      | 3                |

到達時間為 0(依序近乎同時)，求 average waiting time？



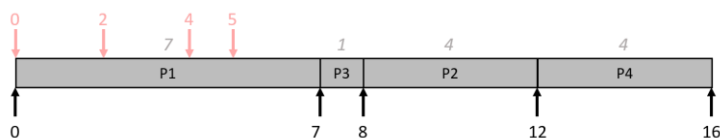
$$\text{Avg waiting time: } [(0-0)+(3-0)+(9-0)+(16-0)]/4 = 7$$

相比於 FCFS： $(0+6+14+21)/4 = 10.25$ ，SJF 顯然較佳許多

例 2：不同時到達之 SJF

| Process | Arrival Time | CPU (burst) Time |
|---------|--------------|------------------|
| P1      | 0            | 7                |
| P2      | 2            | 4                |
| P3      | 4            | 1                |
| P4      | 5            | 4                |

求 average waiting time？



$$\text{Avg waiting time: } [(0-0)+(8-2)+(7-4)+(12-5)]/4 = [0+6+3+7]/4 = 4$$

(三)分析：

1. 排班效益最佳(optimal)，即平均 waiting、turnaround time 最小  
說明：why optimal？因為 Short Job 所減少的 waiting time 必定會大於等於 Long Job 所增加的 waiting time，使得平均 waiting time 為最小
2. 不公平、偏好 Short Job
3. 可能會 Starvation (for Long Jobs)
4. 可以分成：
  - 1.Non-Preemptive：以 SJF 作代表
  - 2.Preemptive：另外叫作 SRTF 法則
5. 較不適合用在 Short-Term Scheduler  
說明：因為 Short-Term Scheduler(舞台助理)執行頻率太高，所以很難在極短的時間間隔內去預估出精確的 CPU burst time for each process，且挑出最小值不易真正呈現出 SJF 之行為，反倒適合用於 Long-Term Scheduler(報名單位)

## 如何預估 Process 之 the next CPU burst time ?

一、公式：(加權指數平均公式)

$$\tau_{n+1} = t_n + (1 - \alpha) * \tau_n$$

$\tau_{n+1}$ ：下次預估值

$t_n$ ：本次實際值

$\tau_n$ ：本次預估值

$\alpha$ ：加權值(介於 0 與 1 之間，包含 0 與 1)

例：( $\alpha = \frac{1}{2}$ )

|     | t0       | t1       | t2       | t3       |          |
|-----|----------|----------|----------|----------|----------|
| 實際值 | 20       | 10       | 45       | 20       |          |
| 估計值 | 20       | 20       | 15       | 30       | 25       |
|     | $\tau_0$ | $\tau_1$ | $\tau_2$ | $\tau_3$ | $\tau_4$ |

二、意義：

若下次的預估值希望拋掉歷史，完全以最近為主，則  $\alpha$  取 1；若想忽略最近值，而是以歷史為主，則  $\alpha$  取 0

## 3. SRTF(Shortest Remaining Time First、又稱 SRJF 或 SRTN(next))

(一)Def：即為”Preemptive SJF”法則，剩餘的 CPU burst Time 最小者，取得 CPU，也就是若新到達之 Process，其 CPU burst 小於目前執行中的 Process 剩的 CPU Time，則新到達之 Process 可以插隊(Preemptive)執行

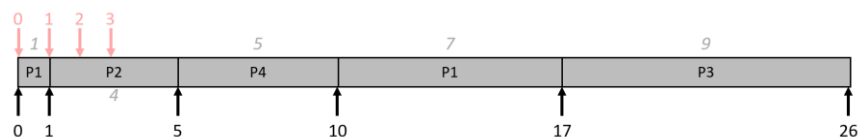
(二)例：

| Process | Arrival Time | CPU (burst) Time |
|---------|--------------|------------------|
| P1      | 0            | 8                |
| P2      | 1            | 4                |
| P3      | 2            | 9                |
| P4      | 3            | 5                |

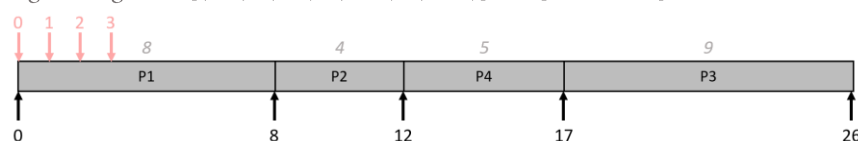
求 average waiting time ?

1. SRTF
2. SJF
3. FIFO

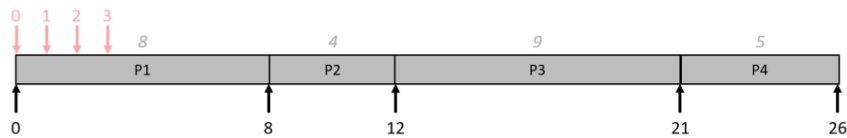
1. Avg waiting time:  $[(0-0)+(10-1)+(1-1)+(17-2)+(5-3)]/4 = [9+0+15+2]/4 = 26/4 = 6.5$



2. Avg waiting time:  $[(0-0)+(8-1)+(17-2)+(12-3)]/4 = [0+7+15+9]/4 = 31/4 = 7.75$



3. Avg waiting time:  $[(0-0)+(8-1)+(12-2)+(21-3)]/4 = [0+7+10+18]/4 = 35/4 = 8.75$



(三)分析：

1. 與 SJF 相比，SRTF 之平均 waiting、turnaround time 會較小，但是付出 Context Switching 負擔較大
2. 不公平、偏好 Short Running-Time Job
3. 會有 Starvation
4. Preemptive 法則

#### 4. Priority

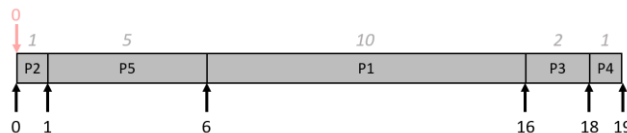
(一)Def：具有 Highest Priority 之 Process 優先取得 CPU，若多個 Process 權值相同，則以 FIFO 為準(也分為 preemptive 與 non-preemptive)

(二)例 1：近乎同時依序到達(the smaller priority number implies higher priority)

| Process | CPU (burst) Time | Priority Number |
|---------|------------------|-----------------|
| P1      | 10               | 3               |
| P2      | 1                | 1               |
| P3      | 2                | 3               |
| P4      | 1                | 4               |
| P5      | 5                | 2               |

求 average waiting time ?

Avg waiting time:  $[(6-0)+(0-0)+(16-0)+(18-0)+(1-0)]/4 = [6+0+16+18+1]/4 = 41/4 = 8.2$

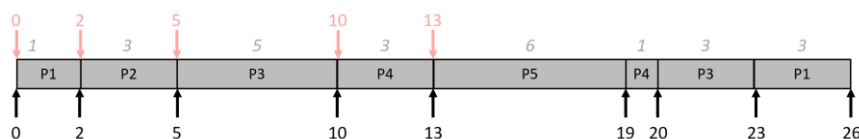


例 2：近乎同時依序到達(the smaller priority number implies higher priority)

| Process | Arrival Time | Priority Number | CPU (burst) Time |
|---------|--------------|-----------------|------------------|
| P1      | 0            | 5               | 5                |
| P2      | 2            | 2               | 3                |
| P3      | 5            | 4               | 8                |
| P4      | 10           | 3               | 4                |
| P5      | 13           | 1               | 6                |

求 average waiting time ?

Avg waiting time:  $[(0-0)+(23-2)+(2-2)+(5-5)+(20-10)+(10-10)+(19-13)+(13-13)]/4 = [21+0+10+6+0]/4 = 37/4 = 9.25$



(三)分析：

1. 是一個具參數化的法則，即給予不同的 **priority** 高低定義，可展現出不同的排班行為。例：

| priority 定義           | 行為   | 包含於關係                   |
|-----------------------|------|-------------------------|
| arrived time 愈小，優先權愈大 | FIFO | FIFO $\subset$ Priority |
| CPU Time 愈小，優先權愈大     | SJF  | SJF $\subset$ Priority  |
| 剩餘時間愈小，優先權愈大          | SRTF | SRTF $\subset$ Priority |

2. 不公平
3. 會有 Starvation(可用 Aging 解決)
4. 分為 Non-Preemptive 或 Preemptive 2 種

## 5. RR(Round Robin)法則

(一)Def：Time-Sharing System 採用

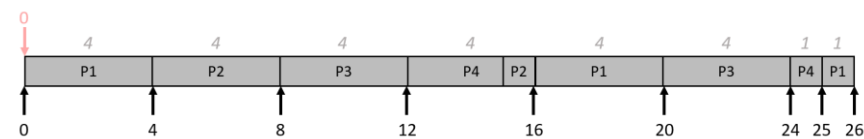
OS 會設定一個『可參數化』的 CPU Time Quantum (or Slice)，當 Process 取得 CPU 執行後，若未能在此 Quantum 內完成工作，則 Timer 會發出 Time-Out Interrupt 通知 OS，OS 會強迫此 Process 放掉 CPU 且回 Ready Queue 中，等待下一輪再取 CPU 執行，每一輪之中，Process 是以 FIFO 排班方式取 CPU(注意要列出 Ready Queue，到達或被踢出都要排到 Ready Queue 的尾端)

(二)例 1：

| Process | CPU (burst) Time |
|---------|------------------|
| P1      | 8                |
| P2      | 4                |
| P3      | 9                |
| P4      | 5                |

求 average waiting time ? 近乎同時依序到達、Time Quantum=4

Avg waiting time:  $[(0-0)+(16-4)+(4-0)+(8-0)+(20-12)+(25-24)+(12-0)+(24-16)]/4 = [12+4+17+20]/4 = 53/4 = 13.25$



例 2：

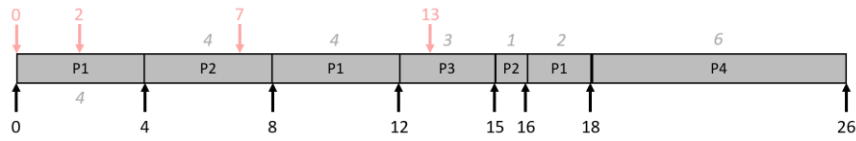
| Process | Arrived Time | CPU (burst) Time |
|---------|--------------|------------------|
| P1      | 0            | 8                |
| P2      | 2            | 4                |
| P3      | 7            | 9                |
| P4      | 13           | 5                |

求 average waiting time ? Time Quantum=4

進入 Ready Queue 順序：



Avg waiting time:  $[(0-0)+(8-4)+(16-12)+(4-2)+(15-8)+(12-7)+(18-13)]/4 = [8+9+5+5]/4 = 27/4 = 6.75$



例 3 :

| Process | Arrived Time | 行為             |
|---------|--------------|----------------|
| P1      | 0            | 5CPU+6IO+4CPU  |
| P2      | 3            | 15CPU          |
| P3      | 7            | 3CPU+10IO+9CPU |
| P4      | 13           | 8CPU           |

求 average waiting time 、turnaround time ? Time Quantum=5

進入 Ready Queue 順序 :

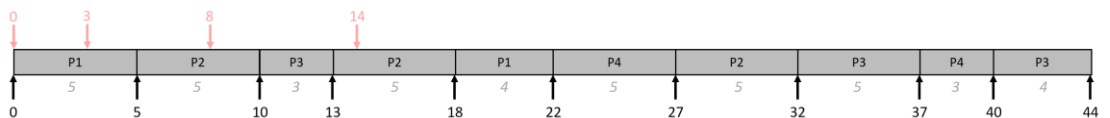
注意：綠色標示為 P1 與 P3 準備好的時候，因為有 IO，所以要將 IO 的時間列入計算考量，至於藍色標示為 Time out 時間，並未包括完工時的切換



Avg waiting time:

$[(0-0)+(18-11)+(5-3)+(13-10)+(27-18)+(10-8)+(32-23)+(40-37)+(22-14)+(37-27)]/4 = [7+14+14+18]/4 = 53/4 = 13.25$

Avg turnaround time:  $[(22-0)+(32-3)+(44-8)+(40-14)]/4 = [22+29+36+26]/4 = 113/4 = 28.25$



從 Ready Queue 中可以看得出來，Context switch 次數：9 次

爭議性的題目：

| Process | Arrived Time | CPU Time |
|---------|--------------|----------|
| P1      | 0            | 10       |
| P2      | 4            | 9        |
| P3      | 8            | 6        |

求 average waiting time ? Time Quantum=4

P1 在第一個 Time Quantum 結束後，要換成誰來執行？是下一個 P1 還是剛到的 P2？

(三)分析：

1. Time-Sharing System 採用、是一個可參數化的法則(ex：Quantum)
2. 公平(FIFO)
3. No Starvation
4. Preemptive 法則(Running to Ready)
5. RR 排班(Mechanism)效益取決於 Time Quantum 大小之制定(Policy)

例 1：Quantum=無限大，則 RR 會退化成 FIFO 法則(排班效能很差)

例 2：Quantum=極小值，則 Context Switching 太頻繁，CPU

utilization 趨近於 0

例 3：依經驗法則，若 Quantum 值能讓 80%的 Jobs 在 Quantum 內完成，效能較佳

Note[補充]Ex：RR 雖然是公平，但也支持差異化(優先權差異)之實現

How do you achieve this?

[法一]：針對高優先權 Process 給予較大的 Quantum 值

[法二]：針對高優先權 Process 在 Ready Queue 中置入多個 PCB pointer 指向此 Process，使得每輪當中，它有多次取得 CPUs 之機會

## 6. Multilevel Queues

(一)Def：

1. 將原本單一條 Ready Queue 變成多條 Ready Queues 且高低優先權不同
2. Queues 之間的排班法則，通常採用”Preemptive Priority”法則
3. 每個 Queue 可以有自已的排班法則。ex：RR 排班法則
4. Process 一旦被置入於某個 Queue 中，就『不可(不允許)』在不同的 Ready Queue 中移動

(二)例：IO Bound 與 CPU Bound，你會置於哪個 Queue?



IO Bound 會置於 Q1；CPU Bound 會置於 Q3：因為 IO Bound 的行程在 CPU 的運算上較短，故可以快速完成後去進行 IO，因此置於 Q1 的平均等待時間會較短；反之 CPU Bound 的 CPU 運算時間較長，置於低優先權 Queue 才不易發生 Convoy Effect(FIFO)

(三)分析：

1. 可參數化的項目眾多，有助於排班設計及效能調校之 flexibility  
ex：
  - (1)Queue 數目
  - (2)Queue 之間的排班法則
  - (3)每個 Queue 自己的排班法則
  - (4)Process 被放在哪個 Queue 之 criteria
2. 不公平
3. 有 Starvation
4. Preemptive

## 7. Multilevel Feedback Queues (MFQs)

(一)Def：與 Multilevel Queue 相似，差別只在於允許 Process 在不同 Queue 之間移動，所以可採取類似”Aging”技術(升級)：每隔一段時間，就將 Process 全部往上移一層 Queue，所以有限的時間後，lowest priority process 會上升到 highest priority queue 中。

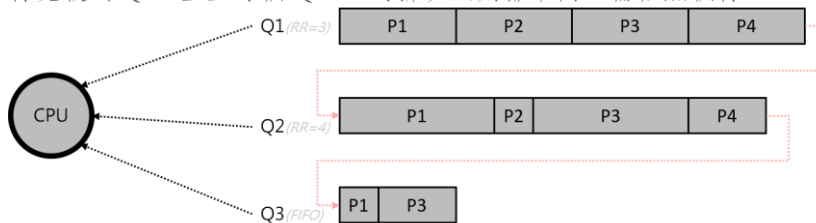
也可搭配降級的作法，來避免 Starvation

(二)例：

| Process | CPU Time |
|---------|----------|
| P1      | 8        |
| P2      | 4        |
| P3      | 9        |
| P4      | 5        |

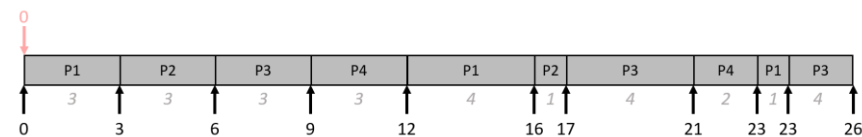
求 average waiting time？近乎同時依序到達

進入 Ready Queues 順序，Time Out 就降級，因此會從 Q1 執行完後到 Q2，再從 Q2 執行完後到 Q3；並且每個 Queue 的排班法則都不同，需依照執行



Avg turnaround time:

$$[(0-0)+(12-3)+(23-16)+(3-0)+(16-6)+(6-0)+(17-9)+(23-21)+(9-0)+(21-12)]/4 = [16+13+16+18]/4 = 63/4 = 15.75$$



(三)分析：

- 可參數化的項目眾多，有助於排班設計及效能調校之 flexibility
  - Queue 數目
  - Queue 之間的排班法則
  - 每個 Queue 自己的排班法則
  - Process 被放在哪個 Queue 之 criteria
- 不公平
- 沒有 Starvation
- Preemptive
- Process 在不同 Queue 之間移動之規則

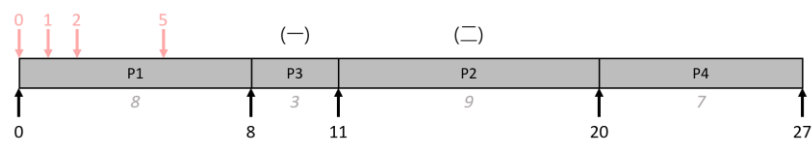
## 8. HRRN(Stalling 版本的補充)

(一)Def : (Highest Response Ratio Next)具有最大的 Response Ratio 值者，優先取得 CPU，而  $RR = (s+w)/s$ ， $s$ =Process Service Time、 $w$  : Process Waiting Time

(二)例：

| Process | CPU Time | Arrival Time |
|---------|----------|--------------|
| P1      | 8        | 0            |
| P2      | 9        | 1            |
| P3      | 3        | 2            |
| P4      | 7        | 5            |

1. 求 P3 的 RR 值？
2. 求 P2 的 RR 值？



1.  $P2 = s+w / s = 9+(8-1) / 9 = 16/9$   
 $P3 = s+w / s = 3+(8-2) / 3 = 3$   
 $P4 = s+w / s = 7+(8-5) / 7 = 10/7$
2.  $P2 = s+w / s = 9+(11-1) / 9 = 19/9$   
 $P4 = s+w / s = 7+(11-5) / 7 = 13/7$

(三)分析：

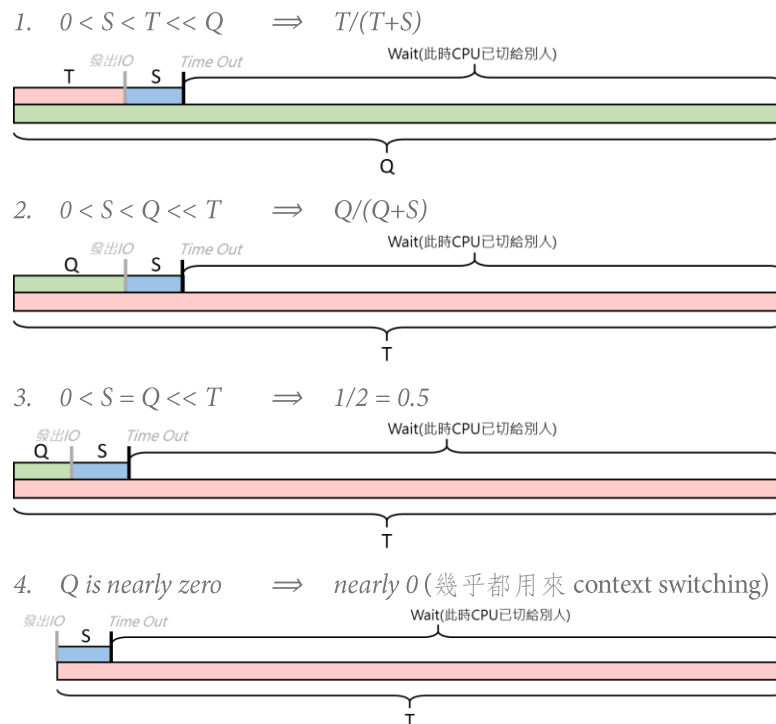
1. 公平
2. 沒有 Starvation
3. Non-Preemptive

小結：

1. 哪些是 Non-Preemptive : FIFO、SJF、Non-Preemptive Priority(、HRRN)
2. 哪些是 No Starvation : FIFO、RR、MFQs(、HRRN)
3. 哪些『包含於關係』是錯的？  
 A :  $FIFO \subset Priority$ 、B :  $SJF \subset Priority$ 、C :  $FIFO \subset RR$   
 D :  $SJF \subset RR$ 、E :  $RR \subset MFQs$   
 $\Rightarrow$  D 是錯的

[補充]CPU Utilization

例 1 : [Modern 版]假設排班 Time Quantum 值=Q，Computer Context Switching Time=S，Process 平均執行每隔 T 時間會發出 I/O request，求下列 cases 之 CPU utilization ？



例 2：[恐][ex]p4-86(50)

10 個 I/O Bound tasks、1 個 CPU Bound task。I/O Bound task 執行每隔 1ms 發出 I/O request，每個 I/O 運作花 1ms 在 Context Switching；Time=0.1ms，求 CPU utilization，以 RR 法則 1.Q=1ms、2.Q=10ms

- 針對 IO Bound task：1 個 IO Bound：1+0.1=1.1 ms  
 針對 CPU Bound task：1+0.1 = 1.1 ms  
 CPU Utilization =  $(1*10+1*1)/(1.1*10+1.1) = 11/12.1 = 91\%$
- $(10*1 + 1*10) / (10*1.1 + 1*10.1) = 20/21.1 = 94.78\%$

**特定 System 之排班設計考量：**

Multi-Processors System 排班設計考量：

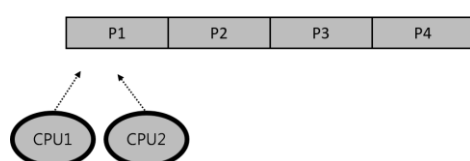
(一)Multi-Processors System 分為 2 型

- ASMP(Master-Slave 架構) $\Rightarrow$ 沒有什麼特殊設計
- SMP：針對 SMP 才有特殊設計，見下：

(二)SMP 排班設計主要有 2 個 Cases(作法)

[法一]：每個 CPU 共享同一條 Ready Queue

當 CPU 完成某 Process 後，就去有剩的 Ready Queue，取走一個 Process 執行。



設計重點：

1. 必提供 Ready Queue 的互斥存取機制(CH6)，若未提供，則可能發生 Process 重複執行，或 Process 被 ignored(無人執行)之錯誤

例：CPU 去取 process 之工作如下：

- (1) 取得(read) ready queue front 端 process 之 PCB pointer
- (2) 刪除此 process pointer from Queue

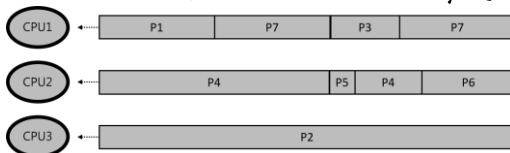
| CPU1                  | CPU2                  |
|-----------------------|-----------------------|
| T1: 取得 P1 PCB pointer | T2: 取得 P1 PCB pointer |
| T3: Dequeue           | T4: Dequeue           |

Race Condition，造成 P1 重複執行，且 P2 沒人執行而被刪

2. 不需考慮負載平衡(Load Balancing issue)

[法二]：每個 CPU 有自己的 Ready Queue

每個 CPU 只會檢查自己的 Ready Queue，有工作就執行，無工作就 idle



設計重點：

1. 不需有互斥存取之考量
2. 反而是考慮負載均衡的議題，避免 CPU 之任務不均(有人很忙、有人 idle)

通常使用 2 種機制來調整 CPU Loading Balance

1. Push Migration：OS 將負擔重之 CPU 工作分給負擔輕之 CPU。ex：領班、工頭
2. Pull Migration(移轉)：閒置的 CPU 把其他非閒置 CPU 的工作拿來(因為 Ready Queue 非共享，所以仍要透過 OS。換言之，idle CPU 是『主動』跟 OS 說，OS 再來分配)。ex：同事愛、革命情感(還是要跟老闆說一下)

Push Migration 與 Pull Migration 兩種方式並不互斥，可以同時存在

一般來說：

1. CPU Bound Process 傾向要 Migrate 到突然有閒置的 Processor 上：可增加 CPU 利用度
2. IO Bound Process 則傾向不 Migrate 至突然有閒置的 Processor：執行時間不一定比較短

Processor Affinity(嗜好)

(一)Def：在 Multi-Processors System 中，當 Process 已決定某 CPU 上執行，則在它執行過程中，儘量不要將它移轉到其他 CPUs 上執行(除非有其必要，

ex：Processor BAD、load balancing...等)，避免 CPU 內之 cache 等內容，要複製(重新佈置 repopulation)且刪除(消去 elimination)之工作影響效能

(二)可以有：

1. Hard Affinity：規定 Process 不可移轉
2. Soft Affinity：盡可能不移轉，若有需要，仍可移轉

## Real-Time System

### Hard Real-Time System :

#### (一)排班設計考量：

步驟：

1. 先確定這些工作是否 Schedulable(可排班化)？[ex：CPU 可以負荷得了？]
2. 確定可 Schedulable 後，再考慮如何排程可以 meet 各自的 Deadline

兩個排班法則：

- (1) Rate-Monotonic Scheduling
- (2) EDF(Earliest Deadline First)法則

#### (二)Schedulable 與否之公式：

如果  $\sum_{i=1}^n \frac{C_i}{P_i} \leq 1$ ，then Schedulable，else Non-Schedulable，其中

n 表 real-time event 之所數目、

$C_i$  表 Event 之所需 CPU Time、

$P_i$  表 Event 之 Period Time 發生週期、

$\frac{C_i}{P_i}$  代表 Event i 對 CPU 之消耗比例

例：有下列四個 Real-Time Event，其 CPU burst Time 分別是 20、50、30、x ms，其 period time 分別是 80、100、300ms 與 1s，則在 Schedulable 前提下，x 不可超過？

$$20/80 + 50/100 + 30/300 + x/1000 \leq 1 \Rightarrow x \leq 150ms$$

#### (三)怎麼排班以滿足各工作 Deadline after Schedule？

### Rate-Monotonic 法則：

(一)採取”Static Priority 且 Preemptive 法則”、Period Time 愈小，優先權愈大

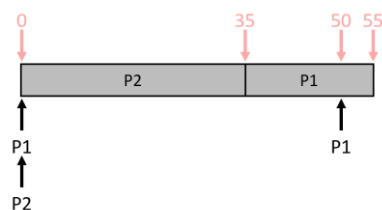
(二)例 1：

| Process | Period Time | CPU Time |
|---------|-------------|----------|
| P1      | 50          | 20       |
| P2      | 100         | 35       |

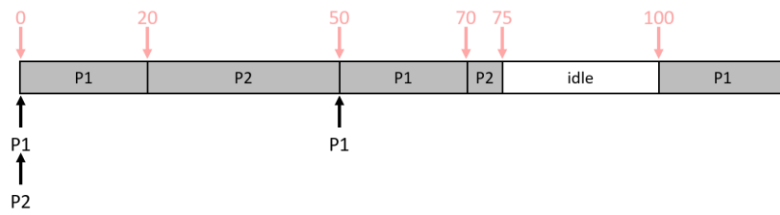
1. 是否 Schedulable?
2. 若規定 P2 優先權最高，且 preemptive，是否滿足 deadline?
3. 採用 Rate-Monotonic 法則，是否滿足 deadline?

1.  $20/50 + 35/100 = 0.75 \leq 1 \Rightarrow \text{Schedulable}.$

2.



3. 因為  $P1$  之  $period\ time = 50 < 100 = P2$  ,  $P1$  優先權大於  $P2$



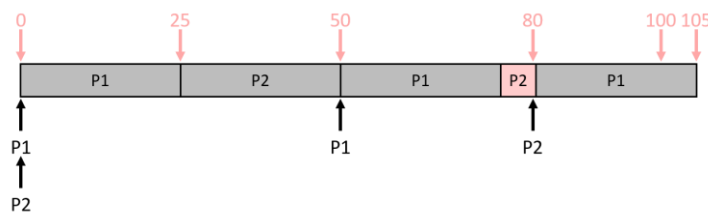
由上圖可知， $P1$ 、 $P2$  皆滿足各自的  $deadline$

例 2：

| Process | Period Time | CPU Time |
|---------|-------------|----------|
| P1      | 50          | 25       |
| P2      | 80          | 35       |

是否 Schedulable?

$25/50 + 35/80 = 1.9375 > 1 \Rightarrow Non-schedulable.$



由上圖可知， $P2$  無法滿足  $deadline$

(三)分析：

1. 並不保證滿足  $Deadline$
2. 在"static" priority 要求下，它是 optimal (若無法滿足  $Deadline$ ，其他 static priority 法則也都無法滿足)

**EDF(Earliest Deadline First)法則：**

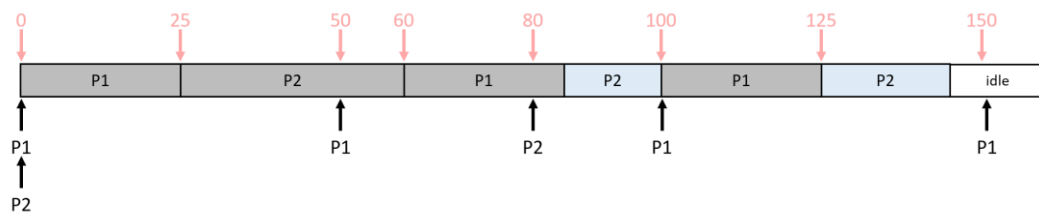
(一)Def：採用"Dynamic" priority 且 preemptive，規定  $Deadline$  愈小(早)、優先權愈高

(二)例：

| Process | Period Time | CPU Time |
|---------|-------------|----------|
| P1      | 50          | 25       |
| P2      | 80          | 35       |

是否 Schedulable ? 過程為何 ? (150 時間單位內)

150 時間單位內，共會有 3 次  $P1$ 、1 次  $P2$  :  $50 < 80$  ;  $100 > 80$  ;  $150 < 160$



藍色為同一  $P2$ ，只是分開 2 次完成  $\Rightarrow P1$ 、 $P2$  皆可即時完工

(三)分析：

1. EDF 保證是 optimal scheduler case (如果工作皆可排)
2. 理論上，CPU utilization 可達 100%，但實際上不可能達 100%，因為有 Context Switching、Interrupt、Handling 等額外負擔

## Soft Real-Time System：

(一)Def：見 CH1

(二)就 CPU Scheduling Design

1. 支援 Preemptive-Priority
2. 不支援“Aging”技術
3. 儘可能地降低 kernel Dispatch Latency Time，使得 Real-Time Process 可以即早開工

(三)降低 kernel Dispatch Latency

1. 困難處(緣由)：大部分的 OS，皆不允許當 kernel 正在執行 System Call 或其他 System Process 時，被 user process 任意插隊(Preemptive)，目的是要確保 kernel data structure 的正確性(不要有 race condition)。然而此種作法對於 Soft Real-Time System 極為不利。Ex：假設目前正在執行一 very long time system call(ex：I/O operation)而此時 real-time process 到達 /fork()，它必須等到 kernel 完成此一 long time system call 後才能取得 CPU

∴Dispatch Latency 太長

∴要解決此一問題，原則是：必須插隊 kernel 且要保障 kernel data structure 之正確性

2. 方法：

1. [法一]：“Preemptive Point”

Def：在 System Call 中加入一些 Preemptive Point(在此時點插隊 kernel 是安全的)，將來，System Call 執行時，若遇到 Preemptive Point System Call 會先暫停，kernel 會檢查是否有 Real-Time Process 存在/到達，若有，則 kernel 繼續執行直到遇到下一個 Preemptive Point

缺點：System Call 中可以加入的 Preemptive Point 數目不夠多，所以 Dispatch Latency 仍然很長

2. [法二]：kernel 可隨時被 Real-Time Process 插隊，但要對於 kernel 共享 Data Structure/Resource 提供嚴謹的互斥存取(Synchronization 機制)，以確保資料之正確性

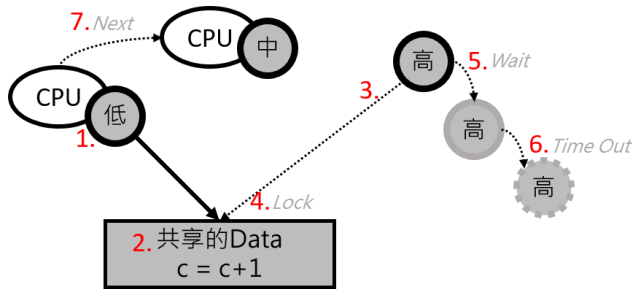
缺點：可能造成優先權反轉(Priority Inversion)

## Priority Inversion(優先權反轉)

Def：高優先權的 Process 所需的共享 Data/ Resource 恰巧被一些低優先權 Process 所把持，無法存取(因為互斥存取控制之故)造成高優先權等待低優先權 Process 之情況(即高要等低，釋放這些共享 Data/Resources)

過程：

1. 低優先權行程取得 CPU 使用權
2. 低優先權行程存取共享資源，且因為其資源非常重要，故有互斥存取
3. 高優先權行程進入、亦想存取共享資源
4. 因低優先權行程已佔有互斥資源，高優先權行程無法存取資源
5. 高優先權行程等待低優先權行程
6. 高優先權行程 Time Out
7. 低優先權行程完成，但高優先權剛好無法執行，故 CPU 切給其他(中優先權)行程

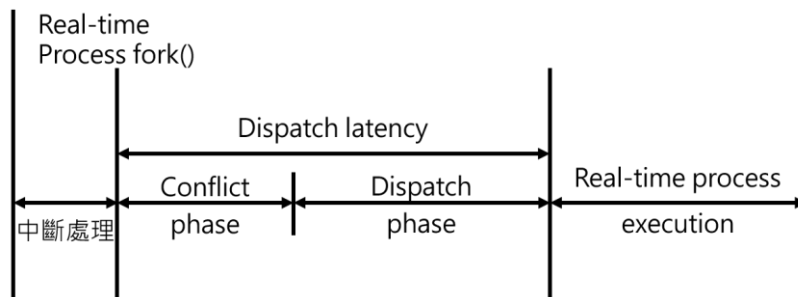


除了以上流程的問題，再加上低優先權 Process 往往無法很快取得 CPU 以完成對共享 Data/Resource 之使用，進而釋放。所以高優先權 Process 被迫要等一段或很久的時間。

解決方式：叫”Priority Inheritance 優先權繼承”

Def：讓低優先權 Process 暫時繼承高優先權之權值，使得低優先權 Process 可以很快取得 CPU，完成對共享 Data/Resources 之使用，並釋放它們，同時也立刻恢復其原本的低權值

Real-Time System 之 Dispatch Latency 組成：



Dispatch Latency 由 2 個 phases 組成

1. Conflict：
  - 甲、Preemptive kernel
  - 乙、低優先權 Release 高優先權所需之 Data/ Resource
2. Dispatch phase：
  - 甲、Context Switching
  - 乙、Change mode to user mode
  - 丙、Jump